VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ
FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

Dokumentácia

Implementácia prekladača imperatívneho jazyka IFJ2024

Tím: xcsirim00
Varianta: vv-BVS

xcsirim00, xgajduv00, xlajdat00, xvalapm00

December 3, 2024

Obsah

1. Úvod 2
1.1. Rozdelenie práce 2
1.2. Rozdelenie bodov 2
2. O vývoji 3
2.1. Proces vývoja 3
3. Návrh a implementácia prekladača 4
3.1. Využité dátové štruktúry a pomocné súbory 4
3.1.1. Dynamický reťazec 4
3.1.2. Zásobník 4
3.1.3. enums.h 4
3.1.4. error_codes.h 4
3.1.5. helpers.c 4
3.2. Lexikálna analýza 5
3.2.1. Token 5
3.3. Syntaktická analýza 6
3.3.1. Syntaktická analýza výrazov 6
3.4. Sémantická analýza 7
3.4.1. Tabuľka symbolov 7
3.5. Generovanie cieľového kódu 8
3.6. AST 8
3.7. Prílohy 9
3.7.1. Konečný automat 9
3.7.2. Legenda ku konečnému automatu 10
3.8. Pravidlá k AST 11
3.8.1. LL gramatika 12
3.8.2. LL tabuľka 14
3.8.3. Precedenčná tabuľka 16
3.9. Viacpriechodový prístup 16

1. Úvod

Zadanie tohoto projektu bolo vytvoriť program v jazyku C, ktorý načíta zdrojový kód z jazyka IFJ24 a preloží ho do jazyka IFJcode24. Nasledujúca dokumentácia pozostáva z našich projektových skúseností, implementačných problémov, riešení a iných detailov. Veríme, že naše skúsenosti a riešenia budú slúžiť ako inšpirácia pre budúce ročníky.

1.1. Rozdelenie práce

- **xcsirim00:**
  - Kapitán tímu
  - Pomocné funkcie
  - Syntaktická analýza výrazov
- **xgajduv00:**
  - Lexikálna analýza
  - Generátor kódu
- **xlajdat00:**
  - Správa repozitáru
  - Sémantická analýza
  - Pomocné štruktúry
- **xvalapm00:**
  - Dokumentácia
  - Syntaktická analýza

1.2. Rozdelenie bodov

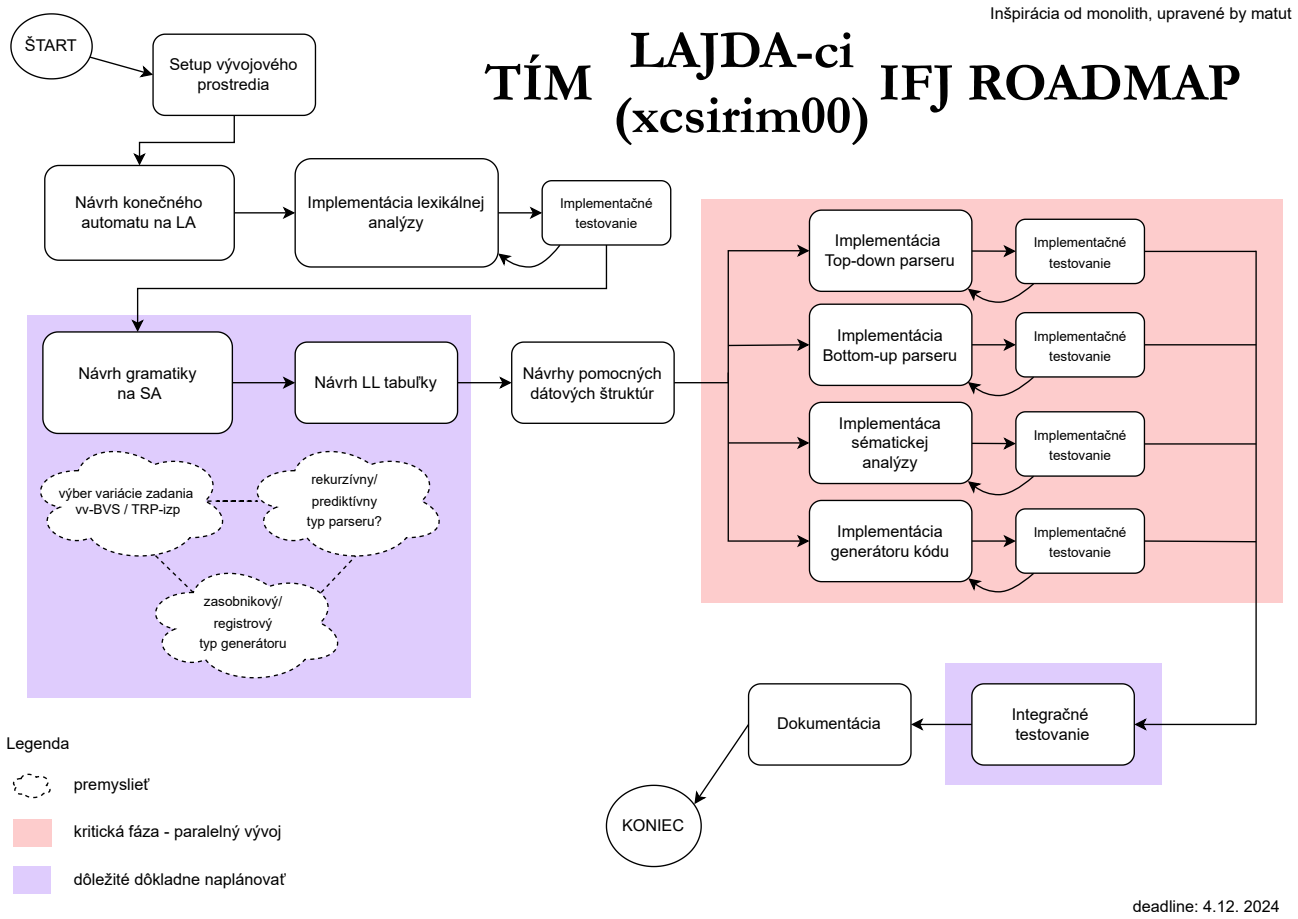
xcsirim00:	27%
xgajduv00:	24%
xlajdat00:	27%
xvalapm00:	22%

2. O vývoji

Vývoj prebiehal vlastnou agilnou metodikou. Stretávali sme sa raz týždenne na štrukturovaných strategických schôdzach, kde sme diskutovali o aktuálnych implementačných detailoch. Podelili sme sa nielen o teoretické, ale aj o praktické vysvetlenie našej práce za uplynulý týždeň a ujasnili sme si celkový stav projektu a naše nadchádzajúce ciele. Schôdze často slúžili aj na skupinové programovanie či brainstorming, kde sme ako tím vymýšľali možné riešenia alebo diskutovali vhodnosť rôznych algoritmov.

Na verzovanie projektu sme použili Git s platformou GitHub. Stratégia verzovania bola paralelný vývoj rôznych častí projektu v príslušných vetvách a ich následné zlúčenie do hlavnej vetvy pri splnení očakávaní funkčnosti. Ako primárny kanál komunikácie sme používali vlastný štrukturovaný Discord server s rôznymi kanálmi na rôzne typy komunikácie a integrovanou aplikáciou sledujúcou GitHub.

2.1. Proces vývoja



3. Návrh a implementácia prekladača

3.1. Využité dátové štruktúry a pomocné súbory

3.1.1. Dynamický reťazec

Dynamický reťazec v `dynamic_string.c` umožňuje spracovanie reťazcov s premenlivou dĺžkou počas lexikálnej analýzy. Slúži na načítanie identifikátorov, literálov a kľúčových slov. Zabezpečuje dynamickú správu pamäte pre reťazce a poskytuje funkcie na pridávanie, porovnávanie a konverziu reťazcov.

3.1.2. Zásobník

Zásobník je využívaný najmä počas syntaktickej analýzy výrazov na správne uchovávanie operandov a operátorov, čo umožňuje spracovanie výrazov podľa pravidiel prednosti. Implementácia zásobníka sa nachádza v súbore `stack.c`.

Zásobník obsahuje ukazovateľ na vrchný prvok, kde každý prvok je reprezentovaný tokenom, ukazovateľ na vlastný AST uzol a ukazovateľ na ďalší element v rade.

3.1.3. enums.h

Súbor obsahuje všetky typy tokenov, typy stavov v konečnom automate a typy rozsahov platnosti.

3.1.4. error_codes.h

Tento súbor obsahuje definície všetkých chybových kódov, ktoré sa používajú na spracovanie chýb v rôznych častiach prekladača. Zahŕňa makro HANDLE_ERROR, ktoré slúži na jednotné spracovanie chýb.

3.1.5. helpers.c

Tu sa nachádzajú všetky potrebné funkcie na prácu s tokenmi - výpis, vytvorenie, skopírovanie hodnôt tokenov.

3.2. Lexikálna analýza

Riešenie lexikálnej analýzy je v súboroch `scanner.c` a `scanner.h`.
Ďalej sa využili dátové štruktúry ako `dynamic_string.c` a `enums.h`.

Pred implementáciou sme sa dohodli na štruktúre **tokenu** a začali s návrhom **konečného automatu**. Pri prvotnom návrhu automatu sme nepočítali so všetkými krajnými prípadmi, a tak sme museli návrh postupne upravovať podľa implementácie. Návrh konečného automatu je zobrazený na diagrame priloženom nižšie: konečný automat.

Lexikálna analýza je implementovaná podľa konečného automatu ako jeden cyklus, kde pre každý stav automatu existuje zodpovedajúca vetva. Ak načítaný znak nepovoľuje prechod do žiadneho ďalšieho stavu automatu a aktuálny stav nie je koncový, je program ukončený s chybovým kódom. Ak je automat v koncovom stave, načítaný znak sa vráti späť na vstup a token sa označí ako platný.

Hlavnou funkciou scannera je funkcia `getNextToken`, ktorá podľa konečného automatu načíta znaky zo vstupu a vracia ich vo forme štruktúry Token.

Na načítanie kľúčových slov sme zvolili postup, pri ktorom sa token najskôr načíta ako identifikátor. Následne ho porovnáme so zoznamom kľúčových slov a ak sa zhoduje s niektorým z nich, priradíme mu príslušný typ a atribút.

3.2.1. Token

Token je definovaný ako štruktúra obsahujúca jeho typ a atribút. Typ určuje, či ide o kľúčové slovo, identifikátor, literál, operátor alebo iný typ. Atribút ako hodnota určená podľa konkrétneho typu tokena.

3.3. Syntaktická analýza

Syntaktická analýza sa nachádza v súboroch `parser.c` a `parser.h`
Využíva štruktúru **Abstraktný syntaktický strom (AST)** implementovaný v `ast.h` a `ast.c`

Pred samotnou implementáciou sme vytvorili gramatiku, ktorá presne definuje pravidlá jazyka IFJ24. Počas tvorby sme narazili na problémy s pravidlami začínajúcimi identifikátormi a s voliteľnou čiarkou za posledným parametrom a argumentom.

Syntaktická analýza bolo implementovaná po vzore **rekurzívne zostupného parsera**, kvôli jeho priamočiarej implementácii skoro perfektne odzrkadľujúcej gramatiku. Nakoniec v implementácii voliteľná čiarka, na rozdiel od identifikátorov problémy nerobila. Implementáciu pravidiel s identifikátormi by značne uľahčila vyrovnávacia pamäť (**buffer**) priamo z lexikálnej analýzy a funkcia nahliadajúca na ďalší token, no vďaka časovým problémom musel byť problém riešený neoptimálne, a to zmenami v syntaktickej analýze výrazov.

Ďalšou kľúčovou časťou syntaktickej analýzy bolo vytvorenie už spomenutého AST. Najprv sme vytvorili príručku s pravidlami, podľa ktorej budú všetky typy inštrukcií reprezentované a ako na seba budú navzájom nadväzovať. Samotnú implementáciu napĺňania AST značne uľahčili pomocné funkcie na zobrazenie tokenov a aj celého AST vrátane jeho ďalších vnorených štruktúr.

3.3.1. Syntaktická analýza výrazov

Syntaktická analýza výrazov je realizovaná v súboroch `epxr_parser.h` a `expr_parser.h`. V implementácii je ďalej použitá dátová štruktúra **Zásobnik**. Taktiež využíva AST na reprezentáciu výrazov.

Algoritmus syntaktického analyzátora výrazov bol založený na metóde „shift-reduce parsing“ a zahŕňa tvorbu precedenčnej tabuľky na porovnanie precedencie prvkov na zásobníku. Tabuľka bola zároveň využitá na zakázanie určitých situácií, ktoré podľa gramatiky výrazov nemôžu nastať, ako napríklad zreťazenie relačných operátorov. Tieto neplatné vzťahy sú v tabuľke reprezentované znakom „x“. Syntaktická analýza výrazov zároveň zabezpečuje vyvážené používanie zátvoriek a kontrolu počtu relačných operátorov.

Počas analýzy sa najprv naplní vstupný zásobník a predbežne sa skontroluje výraz. Následne sa použije kombinácia redukčných pravidiel, pomocného zásobníka a precedenčnej tabuľky na redukciu výrazu, ktorý je reprezentovaný ako AST. Po dokončení sa AST vráti prostredníctvom ukazovateľa z argumentu, spolu s tokenom ukončujúcim výraz pre jeho ďalšiu kontrolu v syntaktickej analýze. Problémy so spracovaním identifikátorov sa prejavili aj v syntaktickej analýze výrazov, kde sme museli implementovať možnosť dodania maximálne dvoch voliteľných argumentov s už spracovanými tokenmi, čo spôsobilo chaotickejší vzhľad kódu.

3.4. Sémantická analýza

Sémantická analýza, implementovaná v `semantic_analysis.h` a `semantic_analysis.c`, funguje rekurzívne cez riadiacu funkciu, ktorá spracuváva jednotlivé uzly v AST. Implementácia začala identifikáciou potrebných funkcií, ktoré sme postupne dopĺňali.

Integrácia syntaktickej a sémantickej analýzy spočiatku prebiehala hladko, no narazili sme na problém s volaním funkcií pred ich deklaráciou. Tento problém si vyžiadal zásadné prepracovanie, vrátane odstránenia časti práce a vytvorenia samostatnej komponenty na spracovanie volaní v ďalšom priechode AST.

Jedným z hlavných problémov bolo správne riešenie návratového typu funkcií. Tento problém sme vyriešili pridaním parametra s názvom funkcie do tabuľky symbolov. Ďalšou výzvou bolo spracovanie premenných s rovnakým názvom v rôznych rámcoch. Tento problém sme zvládli dodatočným priechodom AST, počas ktorého sa premenné premenovávali pri deklarácii v rámci celého podstromu.

Ďalšie výzvy zahŕňali nesprávnu prácu s „compile-time“ konštantami, ktoré sme spočiatku nahrádzali priamo v AST. Problém sme vyriešili pridaním člena do tabuľky symbolov na uchovanie hodnôt konštánt, pričom sa tieto hodnoty nahrádzali len v špecifických prípadoch.

Asi najväčšou výzvou bolo pochopiť princípy typového prevodu a pravidlá pre rôzne operácie. Pôvodne sme implementovali typový prevod podľa štandardov jazyka Zig s menšími úpravami, no toto riešenie sa ukázalo ako nevyhovujúce voči zadaniu. Nakoniec sme zvolili prístup vychádzajúci z diskusného fóra, ktorý plne zodpovedal špecifikácii.

3.4.1. Tabuľka symbolov

Implementácia tabuľky symbolov je zapísaná v súboroch `symtable.h` a `symtable.c`. Ďalej využíva dátovú štruktúru Zásobník.

Tabuľku symbolov sme sa rozhodli implementovať ako **binárny vyhľadávací strom**, konkrétne ako **AVL strom** kvôli jeho vyvažovacím vlastnostiam. Po analýze obsahu symbolov sme implementovali strom a pridali potrebné funkcie na jeho spravovanie.

Rámce sme riešili pomocou zásobníka, pri čom sa pre každý rámec ukladá nová tabuľka symbolov. Po ukončení rámca sa tabuľka uvoľní z pamäte aj zo zásobníka.

3.5. Generovanie cieľového kódu

Generovanie je implementované v súboroch `code_generator.h` a `code_generator.c`

Hlavnou funkciou generátora kódu je funkcia `generateCode`, ktorá postupne prechádza celé AST a generuje inštrukcie podľa jednotlivých uzlov stromu. Tieto inštrukcie sa ukladajú do dynamického pola, ktoré sa následne vypíše na štandardný výstup.

Na začiatku generátor vygeneruje hlavičku vo forme '.IFJcode24' a následne začína generovať inštrukcie pre vstavané funkcie. Potom prechádza samotným AST, kde iteratívne spracúva uzly reprezentujúce definície funkcií a pomocou funkcie `processNode` generuje inštrukcie pre jednotlivé príkazy. Počas generovaní inštrukcií pre výrazy sme využili zásobník, ktorý efektívne funguje s AST pre jednotlivé výrazy a používa počas toho `PostOrder` priechod.

Keďže v podblokoch nie je možné prekryť už definovanú premennú, všetky premenné, vrátane parametrov funkcie, sú deklarované už na začiatku definície funkcie.

V jazyku IFJ24 nie sú podporované globálne premenné, preto nám väčšinou stačil lokálny rámec. Na odovzdávanie parametrov a návratovej hodnoty pri volaní funkcií sme využili dočasný rámec.

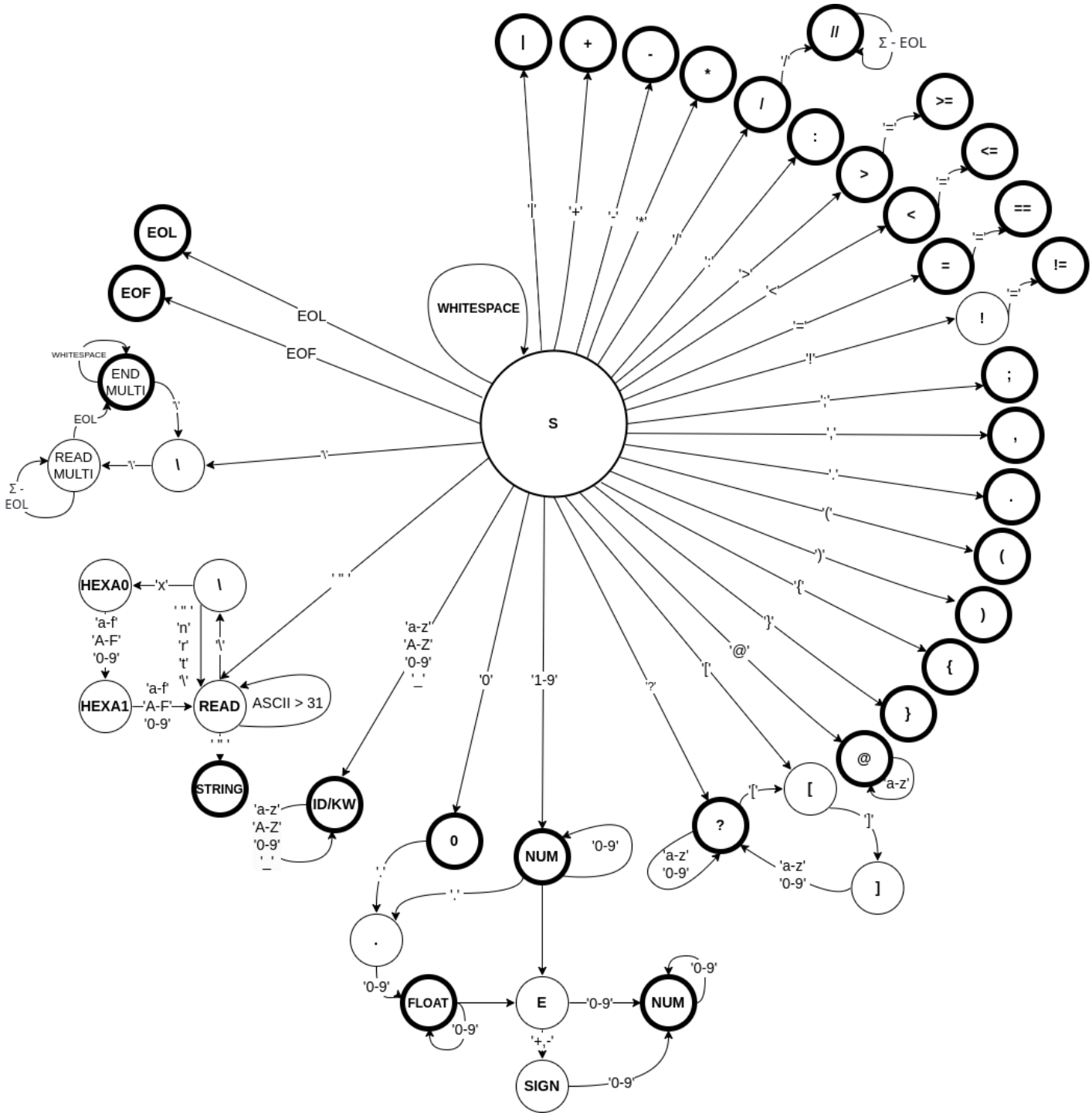
3.6. AST

Na zjednodušenie viacpriechodového prekladu sme využili binárny strom reprezentujúci zdrojový program. Štruktúra stromu obsahovala ukazovateľ na koreň stromu a hodnotu signalizujúcu, že AST reprezentuje výraz. Každý uzol stromu obsahoval ukazovatele na ľavého a pravého potomka, rodiča, token reprezentujúci hodnotu a ukazovateľ na vnorené AST, spolu s hodnotou signalizujúcou, že ide o priradenie do premennej. Uzly AST sa vytvárali počas syntaktickej analýzy a spájali sa pomocou dvoch ukazovateľov, ktoré sa používali ako rodičia pre nové uzly. Na zabezpečenie správnej štruktúry bola vytvorená príručka s referenčnými podstromami, ktoré zobrazovali prevod pravidiel gramatiky do AST.

AST sa ďalej využívalo aj pri sémantickej analýze a aj pri generovaní kódu. Počas sémantickej analýzy sa naplnené AST znova prechádzalo a počas toho sa zisťovalo, či uzly samotné alebo naviazanie viacerých uzlov dodržujú pravidlá. Ak strom prešiel úspešne sémantikou, tak prešiel do generácie kódu a tam sa následne transformoval na cieľový kód.

3.7. Prílohy

3.7.1. Konečný automat

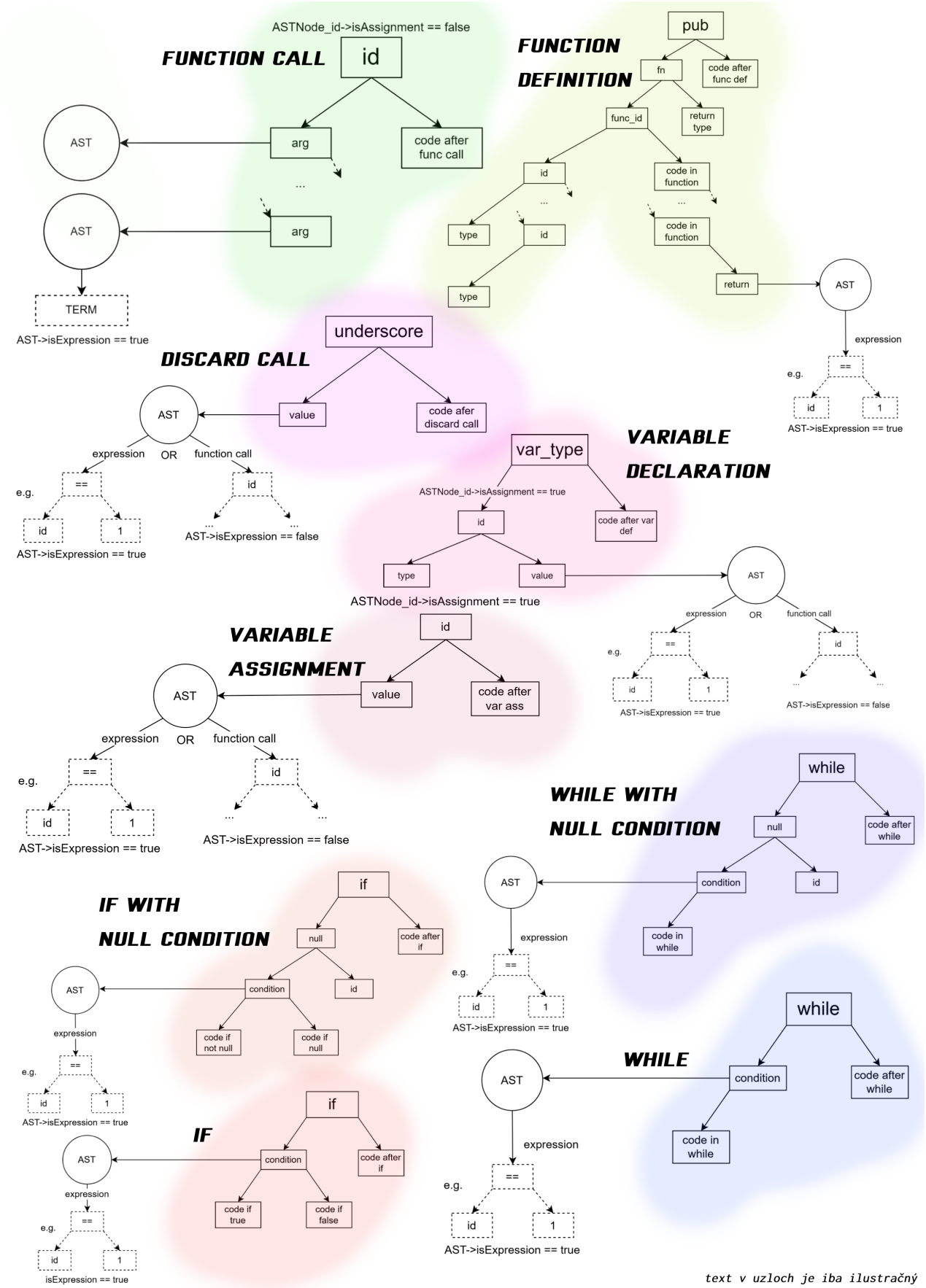


3.7.2. Legenda ku konečnému automatu

Stav	Popis
S	Štart - Počiatočný stav automatu
WHITESPACE	Spracovanie medzier, tabulátorov a iných bielych
EOL	Koniec riadku – stav dosiahnutý po zachytení znaku nového riadku (\n)
EOF	Koniec súboru - stav dosiahnutý na konci vstupného súboru
END MULTI	Koniec viacriadkového reťazca, ukončenie viacriadkového komentára
READ MULTI	Čítanie viacriadkového reťazca, rozpoznávanie textu vo viacriadkových reťazcoch
NUM	Rozpoznávanie celých čísel
FLOAT	Rozpoznávanie desatinných čísel
E	Stav pre exponenciálny zápis čísel
SIGN	Znamienko spracovanie znamienka '+' alebo '-' v exponenciálnom zápise
ID/KW	Spracovanie názvov premenných alebo kľúčových slov
STRING	Reťazec spracovanie textových reťazcov (napr. "text")
HEXA0	Hexadecimálny stav 1 – rozpoznávanie prvého čísla v hexadecimálnom čísle
HEXA1	Hexadecimálny stav 2 – rozpoznávanie druhého čísla v hexadecimálnom čísle
READ	Čítanie hexadecimálnych hodnôt spracovanie znakov (0-9, a-f, A-F)
/	Rozhodovanie medzi delením a začiatkom komentára
//	Spracovanie jednoriadkového komentára
?	Spracovanie nullable typov
@	Spracovanie import (@import(ïfj24.zig")).
!	Spracovanie porovnávacieho operátora (!=)
(,), {, }, [,]	Rozpoznávanie otváracích a zatváracích zátvoriek alebo blokov
=, ==, <=, >=, <, >	Spracovanie operátorov pre priradenie alebo porovnanie
+, -, *	Rozpoznávanie základných aritmetických operátorov.
| ; : . ,	Spracovanie oddelenia výrazov, špecifikácie rozsahov alebo členenia kódu
;	Spracovanie ukončenia príkazov v kóde
:	Spracovanie oddelenia názvu a typu premennej
.	Spracovanie oddeľovača pre prístup k členom štruktúr alebo objektov
,	Oddeľovanie prvkov v parametroch a argumentoch funkcií
|	Začiatok alebo koniec výrazov v if/while s 'nullable' premennými

Tabuľka 1: Legenda ku konečnému automatu

3.8. Pravidlá k AST



text v uzloch je iba ilustračný
xcsirim00

3.8.1. LL gramatika

```
 1. <prog> -> <prolog> <func_defs>
 2. <prolog> -> const ifj = @import("ifj24.zig");
 3. <func_defs> -> <func_def> <func_defs>
 4. <func_defs> -> ε
 5. <func_def> -> pub fn func_id ( <params> ) <func_type>
 6. <func_type> -> <func>
 7. <func_type> -> <v_func>
 8. <func> -> <type> { <statements> <return> }
 9. <v_func> -> void { <statements> <v_return> }
10. <return> -> return <expr> ;
11. <v_return> -> return ;
12. <v_return> -> ε

13. <params> -> id : <type> <next_param>
14. <params> -> ε
15. <next_param> -> , <params>
16. <next_parama> -> ε

17. <type> -> i32
18. <type> -> ?i32
19. <type> -> f64
20. <type> -> ?f64
21. <type> -> []u8
22. <type> -> ?[]u8

23. <statements> -> <statement> <statements>
24. <statements> -> ε

25. <statement> -> <var_def>
26. <statement> -> <var_ass>
27. <statement> -> <if>
28. <statement> -> <while>
29. <statement> -> <func_call>
30. <statement> -> <discard_call>

31. <var_def> -> <var_type> id <type_spec> = <expr> ;
32. <var_type> -> var
33. <var_type> -> const
```

```
34. <type_spec> -> : <type>
35. <type_spec> -> ε

36. <var_ass> -> id = <expr> ;
37. <func_call> -> func_id ( <args> ) ;
38. <func_call> -> ifj . id ( <args> ) ;
39. <discard_call> -> _ = <expr> ;
40. <if> -> if ( <expr> ) <null_cond> { <statements> } <else>
41. <else> -> else { <statements> }
42. <while> -> while ( <expr> ) <null_cond> { <statements> }

43. <null_cond> -> | id |
44. <null_cond> -> ε

45. <args> -> <val> <next_arg>
46. <args> -> ε
47. <next_arg> -> , <args>
48. <next_arg> -> ε
49. <val> -> id
50. <val> -> <expr>
```

Poznámky k gramatike:

- `id` - názov premennej
- `func_id` - názov funkcie - v implementácií vnímaný ako token identifikátoru, rozdiel medzi identifikátorom funkcie a identifikátorom premennej je určený nasledujúcim tokenom otváracej zátvorky. Potrebné tokeny na túto operáciu sú ukladané do bufferu.
- `expr` - výraz

3.8.2. LL tabuľka

	$	const	ifj	=	head	id	pub	fn	id	(	)	void	{	}	return	expr	;
<prog>		1															
<prolog>		2															
<func_defs>	4						3										
<func_def>							5										
<func_type>												7					
<v_func>												9					
<func>																	
<return>															10		
<v_return>														12	11		
<params>						13					14						
<next_param>											16						
<type>																	
<statements>		23	23											24	24		
<statement>		25															
<var_def>		31															
<type_spec>				34													
<var_type>		33															
<var_ass>						36			26								
<func_call>			38														
<discard_call>																	
<if>																	
<else>																	
<while>																	
<null_cond>					43								44				
<args>						45					46					45	
<next_arg>											47						
<val>					49											50	

Tabuľka 2: LL tabuľka 1. časť

Poznámky k LL tabuľke:

- head - @import(ïfj24.zig")

	:	,	i32	f64	[]u8	?i32	?f64	?[]u8	var	func_id	.	_	if	else	while	|
<prog>																
<prolog>																
<func_defs>																
<func_def>		6	6	6	6	6	6									
<func_type>																
<v_func>																
<func>		8	8	8	8	8	8									
<return>																
<v_return>																
<params>																
<next_param>		15														
<type>			17	18	19	20	21	22								
<statements>									23	23		23	23		23	
<statement>									25	29		30	27		28	
<var_def>									31							
<type_spec>	34															
<var_type>									32							
<var_ass>																
<func_call>										37						
<discard_call>												39				
<if>													40			
<else>														41		
<while>															42	
<null_cond>																
<args>																
<next_arg>		47														
<val>																

Tabuľka 3: LL tabuľka 2. časť

3.8.3. Precedenčná tabuľka

	$+$	$-$	$\cdot$	$\div$	$<$	$>$	$\leq$	$\geq$	**!=**	**==**	(	)	**i**	**$**
$+$	>	>	<	<	>	>	>	>	>	>	<	>	<	>
$-$	>	>	<	<	>	>	>	>	>	>	<	>	<	>
$\cdot$	>	>	>	>	>	>	>	>	>	>	<	>	<	>
$\div$	>	>	>	>	>	>	>	>	>	>	<	>	<	>
$<$	<	<	<	<	x	x	x	x	x	x	<	>	<	>
$>$	<	<	<	<	x	x	x	x	x	x	<	>	<	>
$\leq$	<	<	<	<	x	x	x	x	x	x	<	>	<	>
$\geq$	<	<	<	<	x	x	x	x	x	x	<	>	<	>
!=	<	<	<	<	x	x	x	x	x	x	<	>	<	>
==	<	<	<	<	x	x	x	x	x	x	<	>	<	>
(	<	<	<	<	<	<	<	<	<	<	<	x	<	x
)	>	>	>	>	>	>	>	>	>	>	x	>	x	>
i	>	>	>	>	>	>	>	>	>	>	x	>	x	>
$	<	<	<	<	<	<	<	<	<	<	<	x	<	x

Tabuľka 4: Precedenčná tabuľka

3.9. Viacpriechodový prístup

Priechody	**Popis**
1. priechod	Priradenie tokenov a vytvorenie AST
2. priechod	Kontrola sémantických pravidiel v AST
3. priechod	Prechádzanie aktualizovaného AST počas generovania kódu

Tabuľka 5: Tabuľka popisu priechodov

Poznámky k priechodom:

2. priechod - Pri deklarácii premennej sa vykoná priechod celého podstromu a nahradí sa každý výskyt danej premennej za nový názov